lowThrust — how it works

A Kerbal Space Program mod (C#) that flies a low-thrust vessel to an orbit the player types in — about the same body, a moon, another planet or a planet's moon — within a delta-v / time budget. `lowThrust/mod/`.

1. The parts

part	what it does
`core/` (no KSP)	The solver. Two-body maths and Lambert (`Orbit`), the within-SOI law (`Guidance`), the stand-in (`Sim`: a point mass, constant thrust, RK4), targeting and capture (`Transfer`), the phase machine (`Mission`), the dv/time trade (`Tradeoff`), every tunable (`Config`).
`plugin/`	In the game. `Controller` flies the mission. `Game` reads the solar system and the vessel through KSP's orbital elements, runs the engines on rails and sets the warp. `Attitude` turns the vessel with its reaction wheels. `Window` is the toolbar window. `FlightLog` writes one log per flight.
`bridge/` (farm only)	A kRPC service, `LowThrustBridge`, that starts a flight and reports on it, for the farm harness `tools/ltfly.py`. The mod itself needs no kRPC.
`tools/`, `tests/`	`lowthrust.exe mission|fly` (the stand-in on the command line, the trade-off with `--frontier/--max-dv/--max-days`), and the solver's tests (mono).

2. A flight, from the window

- The player picks the body and fills any of periapsis, apoapsis, inclination, LAN and argument of periapsis (blank: free; one apsis alone: circular). Optionally **Max Δv** and **Max time**, and the highest warp the autopilot may use.
- **PLAN** flies the whole mission in the stand-in at `PLAN_GRID` arc widths (section 5) on worker threads and lists the options on the frontier. Clicking one selects it.
- **START**: a Δv limit below `IMPULSIVE_MARGIN` × the impulsive estimate is refused at once. Otherwise the engines are lit, SAS goes off, the throttle is shut, the warp limits are relaxed, and the same scan runs on a worker thread (or the clicked option is used). An option is chosen, or the flight is refused with the cheapest and the fastest that exist. The first leg's `Mission` is built on the worker thread, and the flight begins.
- **TERMINATE** stops it; the vessel is left in physics, warp off, with every relaxed limit restored.

3. Every physics step on rails

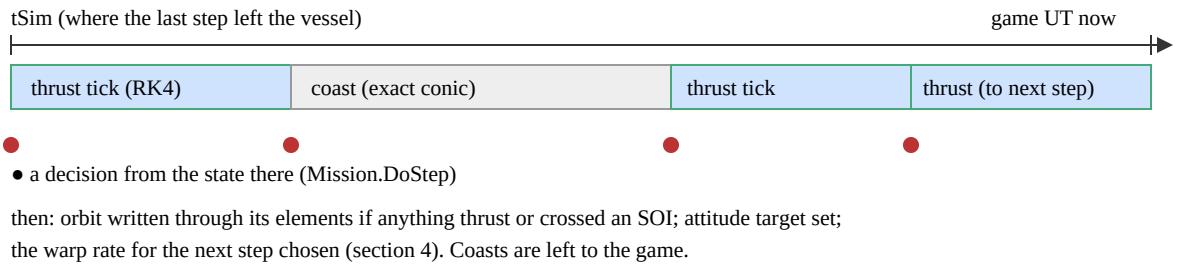


Figure 1. One physics step covers whatever game time the warp gives it. Commands are decided as they fall due, so a step can hold none or many.

- **Read.** If the game's reference body differs from the planner's, the game has crossed an SOI: the planner takes the game's state there, the mission is told (`SoiChange`), and the step ends. Otherwise the state at `tSim` is read from the vessel's orbit (elements → state).
- **Decide** when no command is running: the engines and mass are refreshed, and `Mission.DoStep` returns a throttle, a direction and a tick, or the end of the leg. A coast is run ahead in prediction until the mission next wants thrust, an SOI is near, or `COAST_AHEAD` of a revolution has gone. The whole span becomes one coast command, and the next burn's command is held ready.
- **Thrust command.** It fires only if the thrust axis is within `ALIGN_DEG` of it when it starts. Its direction is the axis's components against the orbit (radial, along-track, normal), held through the tick. RK4 substeps of tick / `THRUST_SUBSTEPS` (gravity plus thrust). Every substep draws the propellants and cuts the thrust to the scarcest one, lowers the mass, and checks for an SOI crossing (then the craft switches body, the mission is told, and the command ends). The delivered dv is booked to the mission when the command ends.
- **Coast command.** The exact conic (`Mission.Coast`), stopping at an SOI boundary.
- **Write.** If anything thrust (or the planner crossed an SOI), the vessel's orbit is set from the planner's state through `Orbit.SetOrbit`.

Frames

The planner works in one right-handed inertial frame per body: x-y the equator, z north, longitudes from KSP's reference direction. States go in and out **only through KSP's orbital elements**, and the bodies are built from their elements, so the planner's LAN is KSP's. World space is used only for pointing, and only as components against the orbit. Each log opens with a `rails check` (bodies 10^6 s ahead against the game's) and a `frame check` (inclination from the elements against the world).

Engines

The lit `ModuleEngines`: vacuum thrust × thrust limiter, Isp at 0 atm, mass flow F/(Isp g_0). Each propellant draws ratio × flow / Σ(ratio × density) units/s; the ratios come from the part's config, so ElectricCharge is paid even if another mod removes it from the module. Draws go through the engine part's flow rules. Availability is the vessel's connected totals.

Pointing

The attitude target is always a set of components against the orbit: a burn's direction, or during a coast the next burn's direction where the coast ends, so the vessel arrives pointed. `Attitude.Step` runs first in the controller's step. Off rails it measures the inertia and the wheels' torque and EC rate. On rails it integrates the rotation in substeps, rate-limited to stop on the target (torque × `MARGIN`, error over `SETTLE`, rate error closed over `RESPONSE`), and draws EC per unit of input. Once within 0.05° with the rate matched, it holds the target exactly as the target turns. A new target that jumps more than 0.05° starts steering again. The rate is handed back to the rigid bodies when the vessel comes off rails. RCS and gimbal are not used.

4. The warp rate

The highest rails rate index such that one physics step (0.02 s × rate of game time):

- keeps the autopilot within `FRAME_BUDGET_MS`: a running average of its time per step lowers the index above the budget and raises it below half;
- draws at most `STEP_RESOURCE_SHARE` of any propellant the vessel holds (for a coast, counted from the coast's end), so generators refill the battery between steps;
- is at most 5 s while the vessel is turning;

- is allowed at the altitude: while flying, every body's rails limits are lowered to the atmosphere top, or the highest terrain + `TERRAIN_MARGIN_M` (`WARP_BELOW_LIMITS`);
- is at most `MAX_WARP_INDEX` and the window's max warp.

Decisions are made inside the physics step; a search (a moon push) holds that one frame.

5. Delta-v against time

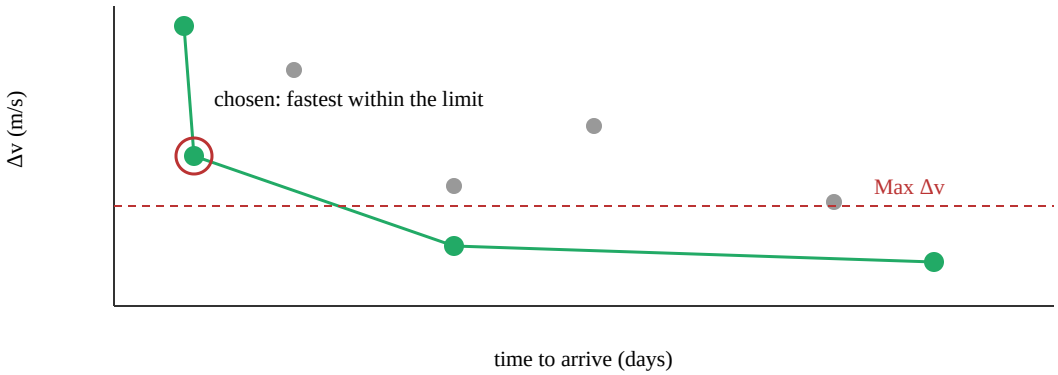


Figure 2. Each point is the whole mission flown in the stand-in at one arc width. Green: the frontier, where no option is both faster and cheaper. Grey: beaten options.

The guidance burns in arcs round each apsis of half-width acos(η). Narrow arcs (η → `ETA_MAX`) are many near-impulsive burns: cheap and slow. Wide ones (→ `ETA_MIN`) finish sooner for more Δv. The time also moves in lumps with waits for a moon or a window. `Tradeoff.Scan` flies the mission at `PLAN_GRID` values of η, even in half-width, in parallel. A flight stops early once it passes `BUDGET_SCAN_FACTOR` × a budget. `Choose` then picks:

budget	option flown
max Δv only	the fastest within it
max days (± max Δv)	the cheapest within both
none	the fastest within `BALANCE_DV_SHARE` of the cheapest
an option clicked	that option
nothing qualifies	refused: the cheapest and the fastest that exist

The chosen η becomes `ETA_DEFAULT` for the flight. The impulsive estimate (`Tradeoff.Impulsive`) is Hohmann legs, departure at periapsis, and capture straight into the target ellipse. In flight the window shows Δv used and days against the plan and the budget; a Δv over budget is logged.

6. The mission's phases

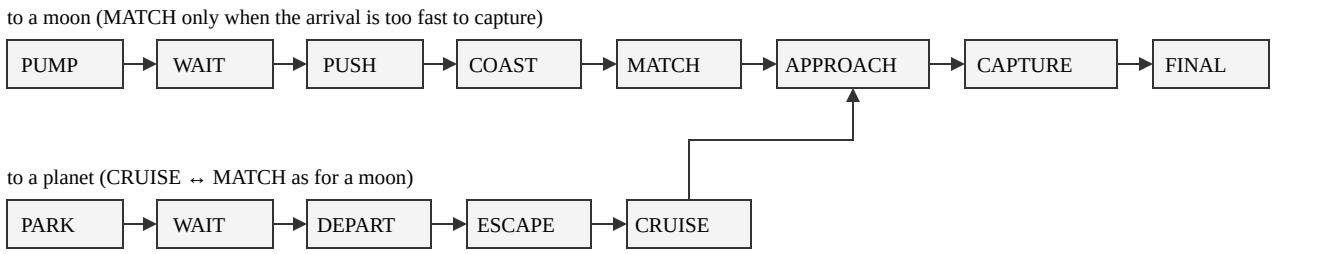


Figure 3. The phases and what follows what. Each is re-decided every tick from the current state.

phase	flies	ends when
PUMP / PARK / FINAL	`Guidance`: prograde/retrograde arcs on a held apse line, each arc fixing the opposite apsis (outward first); plane change at the nodes on the biggest orbit; overlapping burns as one vector. Arc width from η, widened until the remaining changes fit `ARC_PASSES_MAX` passes, or `ETA_WIDE_MIN` while crossing another moon's band. Each arc's first tick flies the burn in prediction and caps it so the next revolution meets no moon; a phasing trim once a revolution if one would.	every targeted apsis, the plane and a held apse are within tolerance → WAIT, or done (FINAL)
WAIT (moon)	Coasts. At each push point (periapsis, or every 30° if no dearer than `PUSH_OFF_PERIAPSIS`), decided ahead, it solves the prograde push whose coast enters the moon's SOI on the aim (`Transfer.PushDv`, the finite burn); it skips a pass that meets another moon or arrives faster than the capture takes.	half the push before its point → PUSH
PUSH	Prograde, the ledger re-solved round its value each tick	< `CORRECT_DV` left → COAST
WAIT / DEPART (planet)	Waits for the periapsis pass nearest the Lambert window (checked along the burn DEPART will fly), then burns prograde to the window's v_∞	v_∞ reached or `DEPART_ARC_DEG` past periapsis → ESCAPE (or WAIT again if still bound)
COAST / CRUISE	B-plane correction toward the aim for the capture periapsis, whenever it needs more than `CORRECT_DV`; a Lambert re-target when the miss is too big to linearise; a moon miss beyond `MISS_REPLAN` → WAIT	SOI entry → APPROACH; the capture law says burn → CAPTURE; MATCH due → MATCH
MATCH	ZEM/ZEV guidance about the parent onto a point outside the SOI, arriving at `ARRIVE_MARGIN` of the fastest capturable speed	matched, `MATCH_END_S` before the aim, or given up → COAST/CRUISE
APPROACH	Nudges the periapsis onto the capture's; dodges a moon on the way in	the capture law says burn → CAPTURE
CAPTURE	Burns centred on periapsis if it fits one pass; otherwise starts once the energy to shed reaches `CAPTURE_PATH_SHARE` of what full thrust removes on the path left, and throttles at that ratio. Retrograde, holding the periapsis band.	bound below the capture apoapsis (or the moon-clear ceiling once past periapsis) → FINAL

A thrusting tick changes the velocity by at most `THRUST_DV_SHARE`. Ticks are uniform in angle (period / `TICKS_PER_ORBIT` at most), and short enough not to jump an SOI boundary.

7. The log

`GameData/LowThrust/PluginData/logs/flight-<date>.log` contains, in order:

- the `config:` line and defaults fingerprint, the rails check, the target, the vessel, the frame check, the propellants, and the `plan:` (the option and the budget);
- the mission's events, and telemetry at most once per `LOG_INTERVAL_UT` game seconds;
- at the end: the result, wall and game seconds by phase, the autopilot's time per step and its slowest decision by phase, thrust ticks fired and not aligned, substeps short of propellant, and the EC the wheels drew.